

Non-Functional Requirements

Document Information

Project: Blogging System API

Version: 1.0

Introduction

This document defines the non-functional requirements of the Blogging System API. These requirements describe the quality attributes, operational characteristics, constraints, and engineering standards that the system must satisfy.

Unlike functional requirements, non-functional requirements focus on system performance, security, maintainability, reliability, scalability, usability, and operational behavior.

Non-Functional Requirement Categories

- Performance
 - Security
 - Reliability
 - Availability
 - Maintainability
 - Scalability
 - Data Integrity
 - Logging & Observability
 - Documentation
 - Coding Standards
-

Performance Requirements

NFR-001 API Response Time

Under normal operating conditions, API endpoints should respond within **2 seconds**, excluding external services such as SMTP.

NFR-002 Efficient Database Queries

Database operations shall use optimized Entity Framework Core queries and avoid unnecessary database calls.

NFR-003 Resource Utilization

The application shall efficiently utilize CPU and memory resources and avoid unnecessary object allocations.

Security Requirements

NFR-004 Password Protection

Passwords shall never be stored or transmitted in plain text.

All passwords shall be hashed using BCrypt before storage.

NFR-005 Secure Authentication

All authenticated requests shall require a valid JWT Access Token.

NFR-006 Refresh Token Security

Refresh Tokens shall be securely generated, stored, and validated before issuing new Access Tokens.

NFR-007 Email Verification

Only verified users shall access protected resources.

NFR-008 Password Reset Security

Password reset OTPs shall:

- Expire after a configurable duration.
 - Be single-use.
 - Become invalid immediately after successful password reset.
-

NFR-009 Input Validation

All incoming API requests shall be validated before business logic execution.

Invalid requests shall return meaningful validation errors.

NFR-010 Authorization

Role-based authorization shall prevent unauthorized access to protected endpoints.

Reliability Requirements

NFR-011 Exception Handling

Unhandled exceptions shall be intercepted by global exception middleware.

The API shall return standardized error responses.

NFR-012 Database Consistency

Database transactions shall maintain data consistency during write operations.

NFR-013 Graceful Failure

Failures in one module shall not unnecessarily affect unrelated components.

Availability Requirements

NFR-014 Continuous Availability

The application shall remain operational unless intentionally stopped for maintenance or deployment.

NFR-015 Database Connectivity

The application shall detect and report database connection failures without crashing unexpectedly.

Maintainability Requirements

NFR-016 Layered Architecture

The application shall follow a Clean Architecture structure with clear separation between:

- API
 - Application
 - Domain
 - Infrastructure
-

NFR-017 Separation of Concerns

Business logic, database access, authentication, validation, and presentation logic shall remain independent.

NFR-018 Dependency Injection

Application services shall be registered and resolved using the built-in ASP.NET Core Dependency Injection container.

NFR-019 Readable Code

Source code shall follow consistent naming conventions, formatting standards, and documentation practices.

NFR-020 Reusable Components

Shared functionality shall be implemented as reusable services rather than duplicated across the application.

Scalability Requirements

NFR-021 Modular Design

The architecture shall allow new modules and features to be added with minimal impact on existing functionality.

NFR-022 Extensibility

The system shall support future enhancements, including:

- Image uploads
- Categories
- Tags
- Likes
- Notifications
- Cloud deployment

without requiring major architectural changes.

Data Integrity Requirements

NFR-023 Referential Integrity

Foreign key relationships shall enforce valid associations between database entities.

NFR-024 Unique Constraints

Unique constraints shall prevent duplicate user email addresses.

NFR-025 Timestamp Accuracy

All entities shall accurately record creation and update timestamps.

Logging and Observability Requirements

NFR-026 OpenTelemetry

The application shall implement OpenTelemetry instrumentation.

NFR-027 Console Export

Telemetry information shall be exported to the application console.

NFR-028 Request Tracing

Incoming HTTP requests shall be traceable throughout the request lifecycle.

NFR-029 Error Logging

Unexpected exceptions shall be logged with sufficient contextual information to support debugging.

Sensitive information such as passwords, JWTs, OTPs, and password hashes shall never be written to logs.

Documentation Requirements

NFR-030 API Documentation

All endpoints shall be documented using Swagger/OpenAPI.

NFR-031 Technical Documentation

Project architecture, database design, authentication flow, and API specifications shall be documented before implementation.

NFR-032 Postman Collection

A Postman collection shall be maintained and updated throughout development.

Coding Standards

NFR-033 Language

The application shall be developed using C#.

NFR-034 Framework

The application shall use ASP.NET Core Web API.

NFR-035 Database

PostgreSQL shall be used as the primary database management system.

NFR-036 ORM

Entity Framework Core shall be used for database access.

NFR-037 Content Format

Blog post content shall be stored as Markdown.

NFR-038 Source Control

All source code shall be maintained using Git.

NFR-039 Testing

Every completed feature shall be tested before implementation of the next feature begins.

Testing shall include:

- Application build verification
 - API endpoint verification
 - Database verification
 - Postman testing
-

NFR-040 Incremental Development

Development shall proceed in clearly defined phases.

Each phase shall be completed, reviewed, and tested before work begins on the next phase.

Non-Functional Requirement Traceability

Every architectural decision, infrastructure component, and implementation strategy shall satisfy one or more Non-Functional Requirement IDs (NFR-001 through NFR-040).

No implementation shall compromise the security, maintainability, reliability, or scalability requirements defined in this document.